# **Computer Architecture** (Practical Class)
## Heterogeneous Data Structures

Luís Nogueira  Nuno Pereira  Raquel Faria

Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

{lmn,nap,arf}@isep.ipp.pt

2018/2019

# Heterogenous data structures in C

In C, we have two ways of combining variables of different types:

- *structures*, declared using the keyword `struct`, aggregate multiple variables under the same name;
- *unions*, declared using the keyword `union`, allow a single variable to use several data types.

Today, we will discuss *structures in C and Assembly*.

- A *structure* is:
  - A continuous memory region (similarly to arrays), composed of different *members*
  - Members are accessed by their name and can have different types
- We can have structures in C by:
  - defining a single structure in a variable;
  - declaring a structure type;
  - declaring a new data type.

# Defining structures in C (1/3)

In the example below, we define a *single structure* in a variable called "c1".

**Structure in C - Defining a structure held by a variable**

```
#include <stdio.h>

/* define a structure in a global variable c1 */
struct  {
  int   n_wheels;
  int   n_passengers;
  float fuel_consumption;
} c1;

int main(){
  /* init and access the data in c1 */
  c1.n_wheels=4;
  c1.n_passengers=2;
  c1.fuel_consumption=12.5;
  printf("The car has %d wheels, %d passengers.Consumption is %f l/100 km.\n",
         c1.n_wheels,c1.n_passengers,c1.fuel_consumption);
  return 0;
}
```

- The structure has only one instantiation (in c1); no other instantiations of the structure exist.

# Defining structures in C (2/3)

In the example below, we declare a *structure type* called "s_car".

**Structure in C - Declaring a structure type**

```
#include <stdio.h>

/* declare a structure type called s_car */
struct s_car {
  int   n_wheels;
  int   n_passengers;
  float fuel_consumption;
} ;

int main(){
  /* define two structures of type car */
  struct s_car c1, c2;
  /* init and access the data in c1 */
  c1.n_wheels=4; c1.n_passengers=2; c1.fuel_consumption=12.5;
  /* init and access the data in c2 */
  c2.n_wheels=5; c2.n_passengers=4; c2.fuel_consumption=20.0;
  printf("Car 2 has %d wheels, %d passengers. Consumption is %f l/100 km.\n",
         c2.n_wheels,c2.n_passengers,c2.fuel_consumption);
  return 0;
}
```

- To instantiate the *car structure*, we define new variables using the name of the structure after the keyword `struct`.

# Defining structures in C (3/3)

In the example below, we declare a new *data type* called "t_car" using the keyword "typedef".

**Structure in C - Declaring a data type for a structure**

```
#include <stdio.h>

/* declare a data type called t_car */
typedef struct { /* we could write 'typedef struct s_car' */
  int   n_wheels;
  int   n_passengers;
  float fuel_consumption;
} t_car; /* this is the name of the structure data type */

int main(){
  /* define two cars */
  t_car c1, c2;
  /* init and access the data in c1 */
  c1.n_wheels=4; c1.n_passengers=2; c1.fuel_consumption=12.5;
  /* init and access the data in c2 */
  c2.n_wheels=5; c2.n_passengers=4; c2.fuel_consumption=20.0;
  printf("Car 2 has %d wheels, %d passengers. Consumption is %f l/100 km.\n",
         c2.n_wheels,c2.n_passengers,c2.fuel_consumption);
  return 0;
}
```

- To instantiate the *car data type*, we define new variables using the name of the data type.

# Pointers to structures in C (1/2)

- We can define pointers to structures

```
struct s_car *ptr;     /* pointer to structure type */
t_car *ptr;            /* pointer to a structure data type  (w/ typedef)
```

- To access the fields of a structure referenced by a pointer, use ->

```
ptr->n_wheels
ptr->n_passengers
ptr->fuel_consumption
```

instead of:

```
(*ptr).n_wheels
(*ptr).n_passengers
(*ptr).fuel_consumption
```

**NOTE:**

- To obtain the address of a structure, use the & operator
- Structures are very similar to arrays (continuous block of memory), ***except* in this aspect** (must use & to get address)

# Pointers to structures in C (2/2)

**Structure in C - Obtaining a pointer to a structure**

```
#include <stdio.h>

typedef struct {
   int   n_wheels;
   int   n_passengers;
   float fuel_consumption;
} t_car;

int main(){
   /* define a car */
   t_car c1;
   /* define a pointer to a car */
   t_car *car_ptr = &c1;

   /* init and access the data in c1 using the pointer */
   car_ptr->n_wheels=4;
   car_ptr->n_passengers=2;
   car_ptr->fuel_consumption=12.5;
   printf("The car has %d wheels, %d passengers. Consum. is %f l/100 km.\n",
           car_ptr->n_wheels,car_ptr->n_passengers,car_ptr->fuel_consumption);

   return 0;
}
```

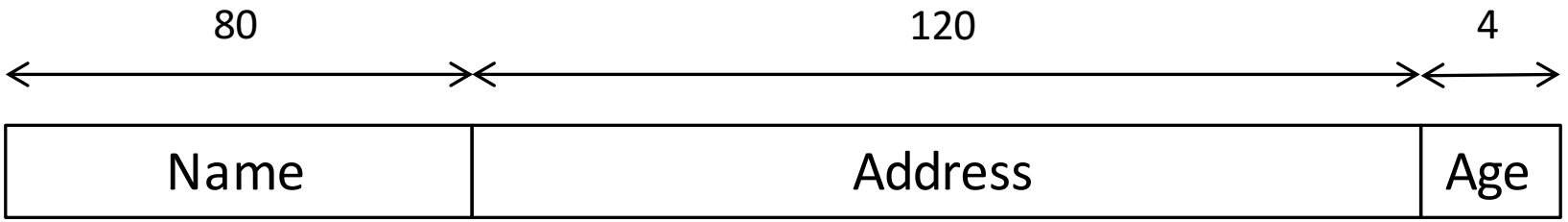
# Storing structures in memory

- Consider the following structure in C

**Example structure in C to store personal data**

```
struct s_person_data{ /* we could also have used typedef ... */
   char name[80];     /* 80 bytes  */
   char address[120]; /* 120 bytes */
   int  age;          /* 4 bytes  */
} ;
```

- To store this structure, we need 204 bytes (80+120+4)




**Important note**

We will see some subtleties of structure sizes in a moment

# Structures in Assembly (1/5)

- In Assembly, we refer to members within a structure by their *offset*
- The offset of each member of the structure refers to the displacement, in bytes, from the start of the structure
- To simplify this access, we can define constants for the offset of each member using the `.equ` directive

```
.equ DATA_SIZE, 204       # total size
.equ NAME_OFFSET, 0       # name is at the beginning of the structure
.equ ADDRESS_OFFSET, 80   # the address starts at byte 80
.equ AGE_OFFSET,200       # age starts at byte 200
```

# Structures in Assembly (2/5)

## In C

```c
void set_age(struct
    s_person_data *pdata)
{
   /* age = 30 */
   pdata->age=30;
}
```

## In Assembly

```
set_age:
    # prologue
    pushl %ebp
    movl %esp, %ebp

    # callee save
    pushl %esi

    # copy pointer of struct to esi
    movl 8(%ebp), %esi

    # age = 30
    movl $30, AGE_OFFSET(%esi)

    # callee restore
    popl %esi

    #epilogue
    movl %ebp,%esp
    popl %ebp
    ret
```

Example: Access to a structure member

**Structure Declaration**

```
struct rec
{
  int x;
  int a[3];
  int *p;
};
```

**Access in C**

```
void set_x(struct rec *r,
           int val)
{
    /* set member x to val */
    r->x=val;
}
```

**Access in Assembly**

```
# %eax = val
movl 12(%ebp),%eax
# %edx = r
movl 8(%ebp),%edx
# Mem[r] = val
movl %eax,(%edx)
```

Memory layout for the structure `rec`

| x | | | a | | | p | | |
|---|---|---|---|---|---|---|---|---|
| 0 | ... | 3 | 4 | ... | 15 | 16 | ... | 19 |

- Note that the address of $x$ (the first member of the structure) is equal to the address of $r$ (the struture)

Example: Obtaining the address of a structure member

**Structure Declaration**

```c
struct rec {
 int x;
 int a[3];
 int *p;
}
```

**Access in C**

```c
int* find_a(struct rec *r,
            int i)
{
    /* return the address
       of member a[i] */
    return &r->a[i];
}
```

**Access in Assembly**

```
# %ecx = i
movl 12(%ebp),%ecx
# %edx = r
movl 8(%ebp),%edx
# %eax = r + 4 + (4 * i)
leal 4(%edx,%ecx,4),%eax
```

We see several constant values in the Assembly code. Why? They do not exist in the C code...

# Structures in Assembly (5/5)

Example: Obtaining a pointer and changing a structure member

**Structure Declaration**

```c
struct rec {
 int x;
 int a[3];
 int *p;
}
```

**Access in C**

```c
void set_p(struct rec *r)
{
  /* return the address
  of member a[i],
  where i is the value
  of member x */
  r->p=&r->a[r->x];
}
```

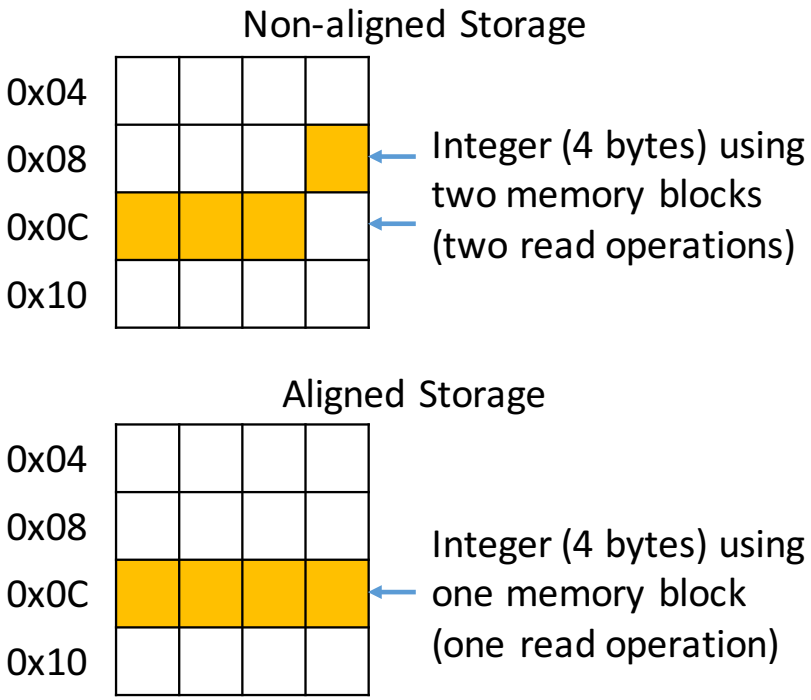
**Access in Assembly**

```asm
# %edx = r
movl 8(%ebp),%edx
# %ecx = r->x
movl (%edx),%ecx
# %eax = r + 4 + 4*(r->x)
leal 4(%edx,%ecx,4),%eax
# r->p = %eax
movl %eax,16(%edx)
```

Based on the size of each member (according to its data type), and the start address of the structure, we must compute the addresses

## Why

- Memory is accessed in blocks of fixed size
  - Usually, 2, 4 or 8 bytes, depending on the system
- Storing data at addresses multiple of the block size — *aligned to the block size* — will reduce the number of memory accesses
- The following example depicts a system accessing memory in blocks of 4 bytes:



# Data alignment in IA32

Data alignment is performed by the compiler

- **The compiler inserts "'spaces' in the data stored in memory to ensure that the members are aligned**
- This improves the performance of the code
- This is required in some architectures; in IA32 it is just recommended
  - in IA32 data can be misaligned, at the cost of a performance penalty

# Data alignment rules in **IA32**

- The alignment rules are expressed as a function of the data size $\mathbb{K}$
  - $\mathbb{K} =$ **1 byte: char**
    - No restrictions
  - $\mathbb{K} =$ **2 bytes: short**
    - The least significant bit must be 0 (that is, the address must be a multiple of 2)
  - $\mathbb{K} =$ **4 bytes: int, long int, float, char *, ...**
    - The 2 least significant bits must be 0 (that is, the address must be a multiple of 4)
  - $\mathbb{K} =$ **8 bytes: double, long long, ...**
    - Windows: The 3 least significant bits must be 0 (that is, the address must be a multiple of 8)
    - Linux: The 2 least significant bits must be 0 (that is, the address must be a multiple of 4); similar to $\mathbb{K} = 4$)

# Data alignment rules in x86-64

- The alignment rules are expressed as a function of the data size $\mathbb{K}$
  - $\mathbb{K}$ = **1 byte: char**
    - No restrictions
  - $\mathbb{K}$ = **2 bytes: short**
    - The least significant bit must be 0 (that is, the address must be a multiple of 2)
  - $\mathbb{K}$ = **4 bytes: int, float, ...**
    - The 2 least significant bits must be 0 (that is, the address must be a multiple of 4)
  - $\mathbb{K}$ = **8 bytes: double, long int, long long, char *, ...**
    - Windows and Linux: The 3 least significant bits must be 0 (that is, the address must be a multiple of 8)

# Data alignment in structures (1/2)

**Particularly important to know about data alignment when dealing with structures in C**

- In order to be able to share structures between C and Assembly, we have to be aware of the data alignment made by the compiler

**Placement of the structure in memory**

- Each structure has an alignment requirement of $\mathbb{K}$, where $\mathbb{K}$ is the size of the largest member of the structure
- The starting address of the structure must be a multiple of $\mathbb{K}$
- The total size of the structure must be a multiple of $\mathbb{K}$

**Inside the structure**

- We must satisfy the alignment requirements for each member of the structure

# Data alignment in structures (2/2)

**Example**: In Linux IA32

- $\mathbb{K}$=4, the member of type `double` is treated the same as members of size 4

```
struct S1 {
    char c;
    int i[2];
    double v;
} *p;
```

| c | 3 bytes | i | v |
|---|---|---|---|
| 0 | | 4 ... 11 | 12 ... 19 |

**Example**: In Windows or IA64

- $\mathbb{K}$=8, due to the member of type `double`

```
struct S1 {
    char c;
    int i[2];
    double v;
} *p;
```

| c | 3 bytes | i | 4 bytes | v |
|---|---|---|---|---|
| 0 | | 4 ... 11 | | 16 ... 23 |

# Practice

- Consider the following data type:

```
typedef struct{
  char age;
  short number;
  int grades[10];
  char name[80];
  char address[120];
}Student;
```

- Develop in Assembly the function `void save_grades(Student *s, int *new_grades)` that that copies all the elements of the array `new_grades` to the field grades of the structure pointed by `s`. Assume that the array `new_grades` has 10 elements